

Лабораторная работа № 4

Тема: Связи между таблицами Internet Shop

Цель работы: Расширить базу shop_db, создать таблицы пользователей и заказов и практически реализовать три типа связей:

- 1:1
- 1:N
- M:N

Используем уже существующие:

- categories
- products

и добавим:

- users
- user_profiles
- orders
- order_items

1. Проверьте существующие таблицы

В базе должны присутствовать:

categories

products

Проверьте:

```
SELECT *
```

```
FROM categories
```

```
ORDER BY id;
```

и:

```
SELECT *
```

```
FROM products
```

```
ORDER BY id;
```

В products должен использоваться:

category_id

с внешним ключом на:

categories.id

Это уже пример связи:

Category 1 → N Products

2. Создайте таблицу users

```
CREATE TABLE users (
```

```
  id INTEGER
```

```
    GENERATED ALWAYS AS IDENTITY
```

```
    PRIMARY KEY,
```

```
  name VARCHAR(100)
```

NOT NULL,

email VARCHAR(150)

NOT NULL

UNIQUE

);

3. Добавьте пользователей

INSERT INTO users

(name, email)

VALUES

('Anna', 'anna@example.com'),

('Ivan', 'ivan@example.com'),

('Maria', 'maria@example.com');

Проверьте:

SELECT *

FROM users

ORDER BY id;

Часть 1. Связь 1:1

4. Создайте таблицу user_profiles

CREATE TABLE user_profiles (

user_id INTEGER

PRIMARY KEY

REFERENCES users(id),

phone VARCHAR(30),

address VARCHAR(250)

);

Обратите внимание:

user_id

одновременно является:

PRIMARY KEY

FOREIGN KEY

5. Добавьте профиль

Узнайте фактический id пользователя Анна.

Затем выполните, например:

INSERT INTO user_profiles

(

user_id,

phone,

address

)

```
VALUES
(
  1,
  '+7 700 111 22 33',
  'Almaty'
);
```

Используйте фактический id.

6. Попробуйте создать второй профиль

```
INSERT INTO user_profiles
```

```
(
  user_id,
  phone,
  address
)
```

```
VALUES
(
  1,
  '+7 777 999 88 77',
  'Astana'
);
```

PostgreSQL должен отклонить запрос.

Причина:

user_id = 1

уже является значением PRIMARY KEY.

Следовательно:

один User → максимум один User Profile

7. Получите пользователя и профиль

```
SELECT
  u.id,
  u.name,
  u.email,
  p.phone,
  p.address
FROM users AS u
LEFT JOIN user_profiles AS p
  ON p.user_id = u.id
ORDER BY u.id;
```

LEFT JOIN позволит увидеть даже пользователей, у которых профиль ещё не создан.

Часть 2. Связь 1:N

8. Создайте таблицу orders

```
CREATE TABLE orders (  
  id INTEGER  
    GENERATED ALWAYS AS IDENTITY  
    PRIMARY KEY,  
  
  user_id INTEGER  
    NOT NULL  
    REFERENCES users(id),  
  
  status VARCHAR(30)  
    NOT NULL  
    DEFAULT 'new',  
  
  created_at TIMESTAMPTZ  
    NOT NULL  
    DEFAULT CURRENT_TIMESTAMP  
);
```

Здесь:

users.id

связан с:

orders.user_id

9. Создайте несколько заказов одного пользователя

Используйте фактический id первого пользователя:

```
INSERT INTO orders  
  (user_id, status)  
VALUES  
  (1, 'new'),  
  (1, 'paid'),  
  (1, 'delivered');
```

10. Создайте заказ другого пользователя

```
INSERT INTO orders  
  (user_id, status)  
VALUES  
  (2, 'new');
```

11. Посмотрите таблицу

```
SELECT *
```

```
FROM orders
```

```
ORDER BY id;
```

В ней должно быть несколько строк с одинаковым:

```
user_id = 1
```

Это нормально.

Один пользователь может иметь много заказов.

12. Получите заказы одного пользователя

```
SELECT
  id,
  user_id,
  status,
  created_at
FROM orders
WHERE user_id = 1
ORDER BY created_at DESC;
```

13. Получите заказы вместе с пользователями

```
SELECT
  o.id AS order_id,
  o.status,
  o.created_at,
  u.id AS user_id,
  u.name AS user_name,
  u.email
FROM orders AS o
JOIN users AS u
  ON u.id = o.user_id
ORDER BY o.id;
```

Здесь реализована связь:
User 1 → N Orders

Часть 3. Проверка FOREIGN KEY

14. Попробуйте создать заказ несуществующего пользователя

```
INSERT INTO orders
  (user_id, status)
VALUES
  (999999, 'new');
```

PostgreSQL должен отклонить запрос.
Причина:
users.id = 999999
не существует.

Часть 4. Связь M:N

15. Определите бизнес-правило

Для Internet Shop:

один Order может содержать много Products
и одновременно:

один Product может входить во много Orders

Следовательно:
Orders M ↔ N Products
Нужна промежуточная таблица.

16. Создайте order_items

```
CREATE TABLE order_items (  
  order_id INTEGER  
    NOT NULL  
    REFERENCES orders(id),  
  
  product_id INTEGER  
    NOT NULL  
    REFERENCES products(id),  
  
  quantity INTEGER  
    NOT NULL  
    CHECK (quantity > 0),  
  
  price INTEGER  
    NOT NULL  
    CHECK (price > 0),  
  
  PRIMARY KEY (  
    order_id,  
    product_id  
  )  
);
```

17. Разберите структуру

order_id

указывает на:

orders.id

А:

product_id

указывает на:

products.id

Одна строка order_items означает:

конкретный Product входит в конкретный Order

18. Посмотрите существующие товары

```
SELECT
```

```
  id,
```

```
  name,
```

```
  price
```

FROM products
ORDER BY id;
Запишите фактические id.

19. Посмотрите существующие заказы

SELECT
id,
user_id,
status
FROM orders
ORDER BY id;
Запишите id первого заказа.

20. Добавьте товары в заказ

Предположим:

order_id = 1

а товары имеют:

product_id = 1

product_id = 2

product_id = 3

Выполните:

INSERT INTO order_items

(
order_id,
product_id,
quantity,
price
)

VALUES

(
1,
1,
1,
15000
)

(
1,
2,
2,
8000
)

(
1,
3,

```
1,  
85000
```

```
);
```

Используйте фактические значения id и цен.

21. Добавьте один товар в другой заказ

Например:

```
INSERT INTO order_items
```

```
(  
  order_id,  
  product_id,  
  quantity,  
  price
```

```
)
```

```
VALUES
```

```
(  
  2,  
  1,  
  1,  
  15000
```

```
);
```

Теперь один и тот же Product связан с несколькими Orders.
Это вторая сторона M:N.

22. Посмотрите промежуточную таблицу

```
SELECT *
```

```
FROM order_items
```

```
ORDER BY order_id, product_id;
```

Пример:

```
order_id | product_id | quantity | price
```

```
-----  
1        | 1          | 1        | 15000  
1        | 2          | 2        | 8000  
1        | 3          | 1        | 85000  
2        | 1          | 1        | 15000
```

23. Получите товары конкретного заказа

```
SELECT
```

```
  p.id,  
  p.name,  
  oi.quantity,  
  oi.price
```

```
FROM order_items AS oi
```

```
JOIN products AS p
```

```
ON p.id = oi.product_id
WHERE oi.order_id = 1
ORDER BY p.name;
```

Результат должен показать только товары заказа 1.

24. Получите заказ с пользователем и товарами

```
SELECT
  o.id AS order_id,
  o.status,
  u.name AS user_name,
  p.name AS product_name,
  oi.quantity,
  oi.price
FROM orders AS o
JOIN users AS u
  ON u.id = o.user_id
JOIN order_items AS oi
  ON oi.order_id = o.id
JOIN products AS p
  ON p.id = oi.product_id
WHERE o.id = 1
ORDER BY p.name;
```

Один результат объединяет данные:

- users
- orders
- order_items
- products

25. Получите заказы, содержащие определённый товар

Выберите существующий product_id.

```
SELECT
  o.id AS order_id,
  o.status,
  o.created_at,
  oi.quantity
FROM order_items AS oi
JOIN orders AS o
  ON o.id = oi.order_id
WHERE oi.product_id = 1
ORDER BY o.created_at DESC;
```

Это показывает связь M:N в обратном направлении.

26. Проверьте составной PRIMARY KEY

Попробуйте второй раз добавить в заказ ту же пару:

```
INSERT INTO order_items
(
    order_id,
    product_id,
    quantity,
    price
)
VALUES
(
    1,
    1,
    5,
    15000
);
```

Если такая пара уже существует, PostgreSQL должен отклонить вставку.

Причина:

PRIMARY KEY (order_id, product_id)

запрещает повтор одинаковой пары.

Количество следует изменять через UPDATE.

27. Измените количество товара

```
UPDATE order_items
```

```
SET quantity = 3
```

```
WHERE order_id = 1
```

```
    AND product_id = 1;
```

Проверьте:

```
SELECT *
```

```
FROM order_items
```

```
WHERE order_id = 1;
```

28. Проверьте ограничение quantity

Попробуйте:

```
UPDATE order_items
```

```
SET quantity = 0
```

```
WHERE order_id = 1
```

```
    AND product_id = 1;
```

PostgreSQL должен отклонить запрос из-за:

```
CHECK (quantity > 0)
```

29. Рассчитайте стоимость позиции

```
SELECT
```

```
    p.name,
```

```
    oi.quantity,
```

```
oi.price,  
oi.quantity * oi.price AS line_total  
FROM order_items AS oi  
JOIN products AS p  
ON p.id = oi.product_id  
WHERE oi.order_id = 1;
```

30. Рассчитайте стоимость заказа

```
SELECT  
order_id,  
SUM(quantity * price) AS total  
FROM order_items  
WHERE order_id = 1  
GROUP BY order_id;
```

Дополнительное задание

Создайте REST endpoint:

GET /api/orders/:id/items

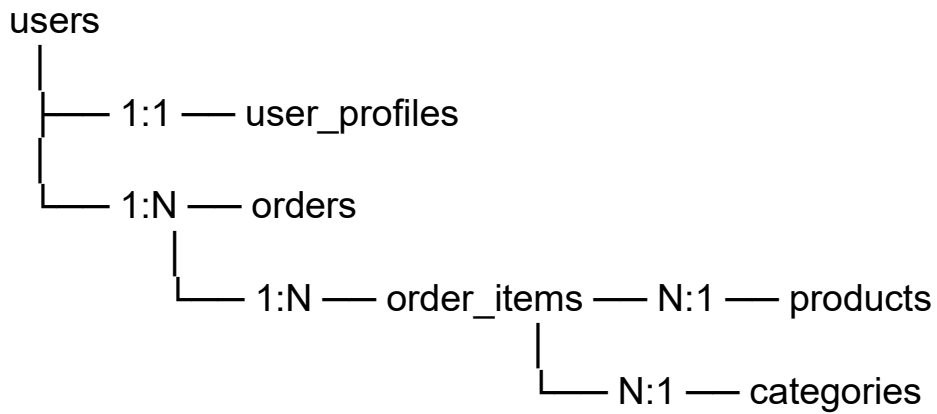
который возвращает товары выбранного заказа.

В orderService.js можно использовать:

```
export async function getOrderItems(  
  orderId  
) {  
  const result =  
    await pool.query(  
      `SELECT  
        p.id,  
        p.name,  
        oi.quantity,  
        oi.price,  
        oi.quantity * oi.price AS "lineTotal"  
      FROM order_items AS oi  
      JOIN products AS p  
      ON p.id = oi.product_id  
      WHERE oi.order_id = $1  
      ORDER BY p.name`,  
      [orderId]  
    );  
  
  return result.rows;  
}
```

Controller должен по-прежнему работать только с HTTP, а SQL находиться в Service.

Итоговая схема



Общая связь:

orders M:N products

реализована через:

order_items

Требования к результату

База должна содержать:

users

user_profiles

categories

products

orders

order_items

Необходимо обеспечить:

- User ↔ User Profile как 1:1;
- User → Orders как 1:N;
- Category → Products как 1:N;
- Orders ↔ Products как M:N;
- внешний ключ user_profiles.user_id;
- внешний ключ orders.user_id;
- внешний ключ products.category_id;
- внешние ключи order_items.order_id и order_items.product_id;
- составной PRIMARY KEY (order_id, product_id);
- CHECK (quantity > 0);
- корректную работу JOIN;
- отклонение несуществующих внешних ключей.